

Generalized Async Return Types in C# 7 with Examples

In this article, I am going to discuss the **Generalized Async Return Types in C# 7** i.e. `ValueTask` with Examples. Please read our previous article where we discuss [Ref Locals and Ref Returns in C# 7](#) with Examples. Before understanding the generalized async return types in C# 7, let's first have a look at asynchronous programming and try to understand how it works.

Return Types of Async Method in C#:

If you have worked with the async methods in C#, then you may know the async methods can have the following return types:

1. **Task<TResult>**: This return type is used when the async method returns a value.
2. **Task**: This return type is used when the async method does not return any value but you want the caller method to wait for the async method execution completion.
3. **void**: This return type is used when the async method does not return any value as well as you don't want the caller method to wait for the async method execution completion.

Let us discuss each of these return types with examples.

Async Method Returning Task<T> in C#

We need to use the **Task<TResult>** return type when the async method is going to return a value after the execution of the method using a return statement. In the following example, the **GetLeisureHours()** async method returns an integer value by using the return statement. So, we specify the **GetLeisureHours()** async method return type as **Task<int>**.

The **ShowTodayInfo()** async method is going to return a string. So, the return type of this async method is **Task<string>**. One more point that you need to remember is whenever you want to call an async method from another async method then you need to use the **await** keyword while calling the method. In our example, we are calling the **GetLeisureHours()** async method from the **ShowTodayInfo()** async method and you can see while calling the **GetLeisureHours()** async method we use the **await** keyword. The **FromResult** async method is a placeholder for an operation that returns a string. The complete example is given below.

```
using System;
using System.Linq;
using System.Threading.Tasks;
namespace GeneralizedAsyncReturnTypesDemo
{
    public class Program
    {
        public static void Main()
        {
            Console.WriteLine>ShowTodayInfo().Result);
            Console.WriteLine("Press any key to exist.");
            Console.ReadKey();
        }
        private static async Task<string> ShowTodayInfo()
```

```

    {
        string ret = $"Today is {DateTime.Today:D}\n" +
                    "Today's hours of leisure: " +
                    $"{await GetLeisureHours()}";
        return ret;
    }
    static async Task<int> GetLeisureHours()
    {
        // Task.FromResult is a placeholder for actual work that returns a
        var today = await Task.FromResult(DateTime.Now.DayOfWeek.ToString(),
        // The method then can process the result in some way.
        int leisureHours;
        //First() method Returns the first element of a sequence
        if (today.First() == 'S')
            leisureHours = 16;
        else
            leisureHours = 5;
        return leisureHours;
    }
}

```

Output:

```

Today is 12 June 2018
Today's hours of leisure: 5
Press any key to exist.

```

For a better understanding of how this happens let's separate the call to GetLeisureHours() async method from the application of await as the following code shows.

```

using System;
using System.Linq;
using System.Threading.Tasks;
namespace GeneralizedAsyncReturnTypesDemo
{
    public class Program
    {
        public static void Main()
        {
            Console.WriteLine>ShowTodaysInfo().Result);
            Console.WriteLine("Press any key to exist.");
            Console.ReadKey();
        }
        private static async Task<string> ShowTodaysInfo()
        {
            var infoTask = GetLeisureHours();
            string ret = $"Today is {DateTime.Today:D}\n" +
                        "Today's hours of leisure: " +
                        $"{await infoTask}";
        }
    }
}

```

```

        return ret;
    }
    static async Task<int> GetLeisureHours()
    {
        // Task.FromResult is a placeholder for actual work that returns a
        var today = await Task.FromResult(DateTime.Now.DayOfWeek.ToString());
        // The method then can process the result in some way.
        int leisureHours;
        //First() method Returns the first element of a sequence
        if (today.First() == 'S')
            leisureHours = 16;
        else
            leisureHours = 5;
        return leisureHours;
    }
}

```

Note: The Result property that we used to retrieve the value is a blocking property. It means if we try to access the value before the async method completes its task, then the thread which is currently active is blocked until the task completes and the value is available. In most real-time applications, we need to access the value by using the “**await**” keyword instead of accessing the property directly. But the point that you need to keep in mind is that you can only use the await property from within an async method.

Async Method Returning Task in C#

We need to use the Task Return Type when the async method is not returning any value after the execution of the method. It means the async method either does not have a return statement in it or it may contain a return statement that doesn't return any value. Such type of async methods returns void if they run synchronously.

If we have an async method with Task return type and if we want our caller method to wait until the async method completes its execution then we need to use the await operator while calling the async method.

In the following example, the **WaitAndApologize()** Async Method return type is Task as it doesn't have a return statement. We are calling this **WaitAndApologize()** async method from the **DisplayCurrentInfo()** async method. As we want to wait until the **WaitAndApologize()** method completes its execution. So, when calling this method from within the **DisplayCurrentInfo()** method we use the await operator.

Again from our Main() method, we are calling the **DisplayCurrentInfo()** async method and our requirement is to wait until the **DisplayCurrentInfo()** method completes its execution. So, here we are using the Wait() method while calling the **DisplayCurrentInfo()** method. We can not use the await operator here because the Main method is not an async method. As we know we can use the await operator only within an async method. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.Threading.Tasks;
namespace GeneralizedAsyncReturnTypesDemo
{

```

```

public class Program
{
    public static void Main()
    {
        //We cannot use await operator as the Main method is not async
        //We cannot call the Result operator as the method is not going to
        anythin
        //So, here we are using the Wait Method
        DisplayCurrentInfo().Wait();

        Console.WriteLine("Press any key to exist.");
        Console.ReadKey();
    }
    //Async Method Return Type is Task means it is not returning anything
    //but we want the caller of this method to wait for its completion
    static async Task DisplayCurrentInfo()
    {
        //Calling the WaitAndApologize() Async Method using await as it is
        method
        await WaitAndApologize();
        Console.WriteLine($"Today is {DateTime.Now:D}");
        Console.WriteLine($"The current time is {DateTime.Now.TimeOfDay:t}");
        Console.WriteLine("The current temperature is 76 degrees.");
    }
    //Again this Async Method Return Type is Task means it is not returning
    //but we want the caller of this method to wait for its completion
    static async Task WaitAndApologize()
    {
        // Task.Delay is a placeholder for actual work.
        await Task.Delay(2000);
        // Task.Delay delays the following line by two seconds.
        Console.WriteLine("\nSorry for the delay. . . .\n");
    }
}

```

Output:

```

Sorry for the delay. . . .

Today is 12 June 2018
The current time is 13:49:09.4593633
The current temperature is 76 degrees.
Press any key to exist.

```

Awaiting the Async Method in Separate Line:

In the below example, within the DisplayCurrentInfo method, first, it will call the **Task task = WaitAndApologize();** method but will not wait for the method to complete its execution. But when we call **await task;** it will wait for the WaitAndApologize method to complete its execution. So, in this way, we

can separate the async method call and await in separate lines. The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.Threading.Tasks;
namespace GeneralizedAsyncReturnTypesDemo
{
    public class Program
    {
        public static void Main()
        {
            //We cannot use await operator as the Main method is not async
            //We cannot call the Result operator as the method is not going to
            anything
            //So, here we are using the Wait Method
            DisplayCurrentInfo().Wait();

            Console.WriteLine("Press any key to exist.");
            Console.ReadKey();
        }
        //Async Method Return Type is Task means it is not returning anything
        //but we want the caller of this method to wait for its completion
        static async Task DisplayCurrentInfo()
        {
            //Separating the calling to WaitAndApologize method from awaiting
            Task task = WaitAndApologize();
            string output = $"Today is {DateTime.Now:D}\n" +
                           $"The current time is {DateTime.Now.TimeOfDay:t}\r"
                           $"The current temperature is 76 degrees.\n";

            //Now it will wait till the task completed by WaitAndApologize method
            await task;
            Console.WriteLine(output);
        }
        //Again this Async Method Return Type is Task means it is not returning
        //but we want the caller of this method to wait for its completion
        static async Task WaitAndApologize()
        {
            // Task.Delay is a placeholder for actual work.
            await Task.Delay(2000);
            // Task.Delay delays the following line by two seconds.
            Console.WriteLine("\nSorry for the delay. . . .\n");
        }
    }
}
```

Async Method Returning Void in C#

We need to use the Void Return Type in C# when the Async Method does not return any value. Then you may have one question in your mind **What is the Difference Between Task and Void Return Types** as both

are going to be used when the Async Method does not return any value?

The difference between them is that if you use the Void Return Type then the async method cannot be awaited. That means the caller of such a method (void return async method) does not have any option to wait for the async method to complete its work. They simply call the async method and continue their work. So, if you have methods other than event handlers that don't return any value, it's always advisable to use Task return type instead of void. On the other hand, if you use Task as the return type of an async method, then you can await the method call i.e. the caller of the method can wait till the async method completes its execution.

Example to Understand Async Method Returning Void in C#

Please have a look at the below example where we are using the async method with void return type. In the below example, from the Main method, we are calling the async SomeMethod method. As the SomeMethod return type is void, so, we cannot use await, result, or Wait. In this example, you will observe that the Main method execution started and was completed before the completion of the SomeMethod.

```
using System;
using System.Threading.Tasks;
namespace GeneralizedAsyncReturnTypesDemo
{
    public class Program
    {
        public static void Main()
        {
            Console.WriteLine("Main Method Started");
            //Here, you cannot use await, result or wait the SomeMethod return
            void SomeMethod();
            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
        //This is an Async Method with Return Type Void
        private static async void SomeMethod()
        {
            Console.WriteLine("SomeMethod Started");
            await Task.Delay(3000);
            Console.WriteLine("SomeMethod Completed");
        }
    }
}
```

Output:

```
Main Method Started
SomeMethod Started
Main Method Completed
SomeMethod Completed
```

I hope now you have some idea regarding the Async Method in C#. So, let us move to the main topic of this article i.e. **Generalized Async Return Types in C# 7**.

Understanding Generalized Async Return Types in C# 7

As of now, we have discussed the Async Method with Return Type **Task**, **Task<T>**, and **Void**. The most important point that you need to keep in mind is that the Task is a class. We also know the reference types behave differently in C#. In some situations, it is better to return anything rather than a Task i.e. rather than a class.

The Generalized Async Returns Types in C# 7 means, now you can return a lightweight value type instead of a reference type (Task) to avoid additional memory allocations. From C# 7, there is an inbuilt value type i.e. **ValueTask<T>** which can be used instead of **Task<T>**.

Note: The .NET Framework provides the **System.Threading.Tasks.ValueTask<TResult>** as a lightweight implementation of a generalized task-returning value. To use the **System.Threading.Tasks.ValueTask<TResult>** type, you must add the **System.Threading.Tasks.Extensions** NuGet Package to your project.

Example to Understand Generalized Async Return Types (ValueTask) in C# 7

Let us understand the **Generalized Async Return Types (ValueTask) in C# 7** with an example. First of all, you need to add the **System.Threading.Tasks.Extensions** package from NuGet. Then modify the Program class code as follows. As you can see in the below example, instead of using **Task<T>**, now we are using **ValueTask<T>** which is a value type, not a reference type and because of this it will have less memory consumption and provides better performance as compared to Task<T> in C#. The following example code is self-explained, so please go through the comment lines for a better understanding.

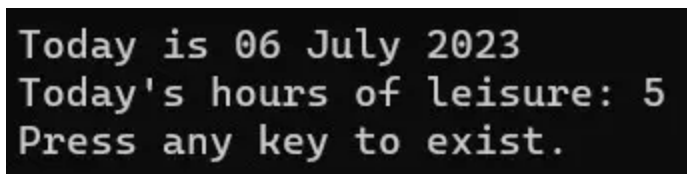
```
using System;
using System.Linq;
using System.Threading.Tasks;
//You need to add System.Threading.Tasks.Extensions Package from NuGet
namespace GeneralizedAsyncReturnTypesDemo
{
    public class Example
    {
        public static void Main()
        {
            Console.WriteLine>ShowTodaysInfo().Result);
            Console.WriteLine("Press any key to exist.");
            Console.ReadKey();
        }
        //Return Type of the Async Method is ValueTask<string> which is a value type
        private static async ValueTask<string> ShowTodaysInfo()
        {
            var infoTask = GetLeisureHours();
            // You can do other work that does not rely on integerTask before
            string ret = $"Today is {DateTime.Today:D}\n" +
                "Today's hours of leisure: " +
                $"{await infoTask}";
        }
    }
}
```

```

        return ret;
    }
    //Return Type of the Async Method is ValueTask<int> which is a value type
    static async ValueTask<int> GetLeisureHours()
    {
        // Task.FromResult is a placeholder for actual work that returns a value
        var today = await Task.FromResult<string>(DateTime.Now.DayOfWeek.ToString());
        // The method then can process the result in some way.
        int leisureHours;
        if (today.First() == 'S')
            leisureHours = 16;
        else
            leisureHours = 5;
        return leisureHours;
    }
}

```

Output:



```

Today is 06 July 2023
Today's hours of leisure: 5
Press any key to exist.

```

You may be thinking that we are talking about the term generalized async, but here we are using only **ValueTask<T>**. So, I would like to clarify your doubt that you can also create your own type which can be the return type of your async method. However, if you do not want to create your own type, then you can use the **ValueTask<T>** which is already available.

In the next article, I am going to discuss the [Expression Bodied Members in C#](#) with Examples. Here, in this article, I try to explain **Generalized Async Return Types in C# 7 i.e. ValueTask** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Generalized Async Return Types in C# 7 i.e. ValueTask with Examples article.